Les algorithmes de recherche et de tri

Algorithmique et Programmation

MAZZA Azzouz

3 juin 2026

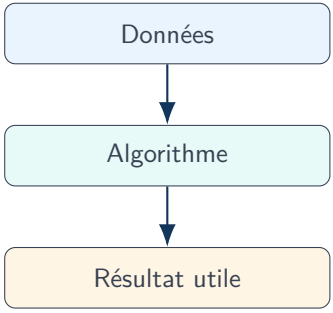
CPGE Oujda – MPSI 2–3

Pourquoi apprendre ces algorithmes ?

Situation-problème

Comment un ordinateur retrouve-t-il rapidement une information parmi des milliers, voire des millions de données ?

- rechercher un fichier ;
- trouver un contact ;
- classer un tableau ;
- interroger une base de données ;
- traiter des données en intelligence artificielle.



Objectifs du chapitre

À la fin de cette séance, vous devez savoir :

- expliquer le principe d'une recherche séquentielle ;
- expliquer le principe d'une recherche dichotomique ;
- implémenter trois tris simples en Python ;
- comparer les algorithmes selon leur complexité ;
- choisir une méthode adaptée à une situation donnée.

Algorithmes de recherche

Recherche séquentielle : idée générale

Principe

On parcourt la liste élément par élément jusqu'à trouver la valeur cherchée ou atteindre la fin de la liste.

Complexité

Dans le pire cas, on examine tous les éléments :

$$\boxed{O(n)}$$



Recherche séquentielle : code Python

```python
def recherche_sequentielle(L, val):
    n = len(L)
    for i in range(n):
        if L[i] == val:
            return True
    return False
```

À retenir

Simple à écrire, utilisable même si la liste n'est pas triée.

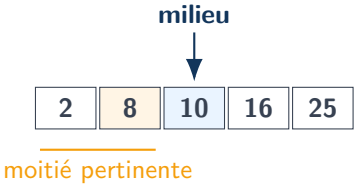
Recherche dichotomique : diviser pour régner

Principe

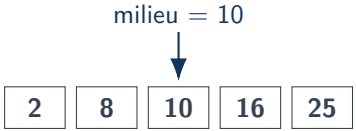
Sur une liste triée, on compare la valeur cherchée avec l'élément du milieu, puis on élimine une moitié de la liste.

Condition essentielle

La liste doit être triée.



Recherche dichotomique : exemple pas à pas



On cherche 8. Comme $8 < 10$, on garde la partie gauche.

Recherche dichotomique : exemple pas à pas



La valeur 8 est trouvée rapidement.

Recherche dichotomique : code Python

```python
def recherche_dichotomique(L, val):
    debut, fin = 0, len(L) - 1
    while debut <= fin:
        milieu = (debut + fin) // 2
        if L[milieu] == val:
            return True
        elif L[milieu] > val:
            fin = milieu - 1
        else:
            debut = milieu + 1
    return False
```

Comparaison des recherches

Algorithme	Liste triée ?	Principe	Complexité
Séquentielle	Non	Parcours complet	$O(n)$
Dichotomique	Oui	Division par 2	$O(\log n)$

Exemple

Pour $n = 1024$ éléments : recherche séquentielle ≈ 1024 tests, recherche dichotomique ≈ 10 tests.

Algorithmes de tri

Le problème du tri

Définition

Trier consiste à organiser une collection d'objets selon un ordre déterminé : ordre numérique, ordre alphabétique, ordre chronologique, etc.



liste initiale liste triée

Tri à bulles

Principe

On compare deux éléments adjacents et on les échange s'ils sont dans le mauvais ordre.

Complexité

$$O(n^2)$$



Exemple complet : tri à bulles

Liste initiale

$$L = [25,\ 16,\ 8,\ 10,\ 2]$$

Passage	Comparaisons principales	Résultat après le passage
1	$25 > 16,\ 25 > 8,\ 25 > 10,\ 25 > 2$	$[16,\ 8,\ 10,\ 2,\ 25]$
2	$16 > 8,\ 16 > 10,\ 16 > 2$	$[8,\ 10,\ 2,\ 16,\ 25]$
3	$8 < 10,\ 10 > 2$	$[8,\ 2,\ 10,\ 16,\ 25]$
4	$8 > 2$	$[2,\ 8,\ 10,\ 16,\ 25]$

Idée clé

À chaque passage, le plus grand élément de la partie non triée remonte vers la droite.

Tri à bulles : code Python

```python
def tri_a_bulles(L):
    n = len(L)
    for i in range(n - 1, 0, -1):
        for j in range(i):
            if L[j] > L[j + 1]:
                L[j], L[j + 1] = L[j + 1], L[j]
```

Tri par sélection

Principe

À chaque étape, on sélectionne le plus petit élément restant et on le place à sa position définitive.

Complexité

$$O(n^2)$$



Exemple complet : tri par sélection

Liste initiale

$$L = [25,\ 16,\ 8,\ 10,\ 2]$$

Étape	Minimum de la partie non triée	Liste obtenue
1	2	[2, 16, 8, 10, 25]
2	8	[2, 8, 16, 10, 25]
3	10	[2, 8, 10, 16, 25]
4	16	[2, 8, 10, 16, 25]

Idée clé

La partie gauche devient progressivement triée et ne sera plus modifiée.

Tri par sélection : code Python

```
def tri_selection(L):
    n = len(L)
    for i in range(n):
        ind_min = i
        for j in range(i + 1, n):
            if L[j] < L[ind_min]:
                ind_min = j
        L[i], L[ind_min] = L[ind_min], L[i]
```

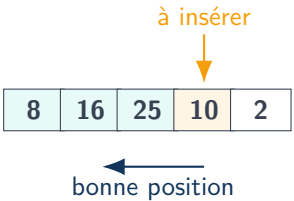
Tri par insertion

Principe

On insère chaque nouvel élément à sa place dans la partie déjà triée.

Analogie

C'est la méthode utilisée naturellement pour trier des cartes à jouer.



Exemple complet : tri par insertion

Liste initiale

$$L = [25,\ 16,\ 8,\ 10,\ 2]$$

Étape	Élément inséré	Liste obtenue
1	16	[16, 25, 8, 10, 2]
2	8	[8, 16, 25, 10, 2]
3	10	[8, 10, 16, 25, 2]
4	2	[2, 8, 10, 16, 25]

Idée clé

À chaque étape, on insère l'élément courant dans la partie gauche déjà triée.

Tri par insertion : code Python

```python
def tri_insertion(L):
    n = len(L)
    for i in range(1, n):
        elt = L[i]
        j = i - 1
        while j >= 0 and L[j] > elt:
            L[j + 1] = L[j]
            j = j - 1
        L[j + 1] = elt
```

Tri par comptage

Principe

Le tri par comptage ne compare pas directement les éléments. Il compte le nombre d'occurrences de chaque valeur, puis reconstruit la liste triée.

Condition importante

Il est adapté lorsque les valeurs sont des entiers dans un intervalle limité.



Liste initiale



Liste triée

Exemple pas à pas : tri par comptage

Liste initiale

$$L = [4,\ 2,\ 2,\ 8,\ 3,\ 3,\ 1]$$

Valeur	0	1	2	3	4	5	6	7
8								
Comptage	0	1	2	2	1	0	0	0
1								

Reconstruction

On écrit chaque valeur autant de fois que son compteur :

$$1,\ 2,\ 2,\ 3,\ 3,\ 4,\ 8$$

Tri par comptage : déroulement

Étape	Action	Résultat
1	Chercher la valeur maximale	$\max(L) = 8$
2	Créer un tableau de compteurs	$C = [0, 0, 0, 0, 0, 0, 0, 0, 0]$
3	Compter les occurrences	$C = [0, 1, 2, 2, 1, 0, 0, 0, 1]$
4	Reconstruire la liste	$[1, 2, 2, 3, 3, 4, 8]$

Complexité

Si n est la taille de la liste et k la plus grande valeur, alors la complexité est :

$$O(n + k)$$

Tri par comptage : code Python

```python
def tri_comptage(L):
    if len(L) == 0:
        return []

    m = max(L)
    compte = [0] * (m + 1)

    # Comptage des occurrences
    for x in L:
        compte[x] = compte[x] + 1

    # Reconstruction de la liste triee
    resultat = []
    for valeur in range(m + 1):
        for _ in range(compte[valeur]):
            resultat.append(valeur)

    return resultat
```

Comparaison des tris simples

Algorithme	Principe	Pire cas	Remarque
Tri à bulles	échanges adjacents	$O(n^2)$	simple mais lent
Tri par sélection	minimum successif	$O(n^2)$	peu d'échanges
Tri par insertion	insertion progressive	$O(n^2)$	efficace si presque trié
Tri par comptage	comptage des valeurs	$O(n+k)$	rapide si k est petit

Activités et synthèse

Activité interactive

Défi 1

Dans la liste triée suivante, trouver 73 par recherche dichotomique :

$$2,\ 5,\ 8,\ 14,\ 18,\ 23,\ 31,\ 42,\ 56,\ 73,\ 91$$

Combien de comparaisons sont nécessaires ?

Défi 2

Trier la liste suivante par insertion :

$$25,\ 16,\ 8,\ 10,\ 2$$

Défi 3

Appliquer le tri par comptage à :

$$4,\ 2,\ 2,\ 8,\ 3,\ 3,\ 1$$

Applications réelles

- moteurs de recherche ;
- bases de données ;
- tableurs ;
- plateformes commerciales ;
- intelligence artificielle ;
- GPS et cartographie ;
- réseaux sociaux ;
- analyse de données.

Synthèse finale

Recherche

- Séquentielle : $O(n)$
- Dichotomique : $O(\log n)$

Tri

- Bulles : $O(n^2)$
- Sélection : $O(n^2)$
- Insertion : $O(n^2)$
- Comptage : $O(n + k)$

Merci.

Questions ?

